Local-First Knowledge Platform for Technical Teams

PROJECTRAG

Answers with evidence, not guesswork

THE CORE PROMISE

Retrieval — Semantic document search across all internal knowledge
Graph — Relationship reasoning over dependencies, services, and topology
Memory — Historical context from prior analyses and validated facts
Validation — Evidence-checked answers with citations and confidence signals

FROM SCATTERED DOCS TO RELIABLE KNOWLEDGE

Inputs: Architecture notes, runbooks, topology docs, incident reports, internal project files

Processing: Ingest → Chunk → Embed → Connect facts across documents

Validation: Cross-check retrieved evidence, reduce hallucination, verify grounding

Output: Grounded answer with supporting evidence and citations

LOCAL-FIRST INGESTION

Sensitive data stays on controlled infrastructure — no external API calls

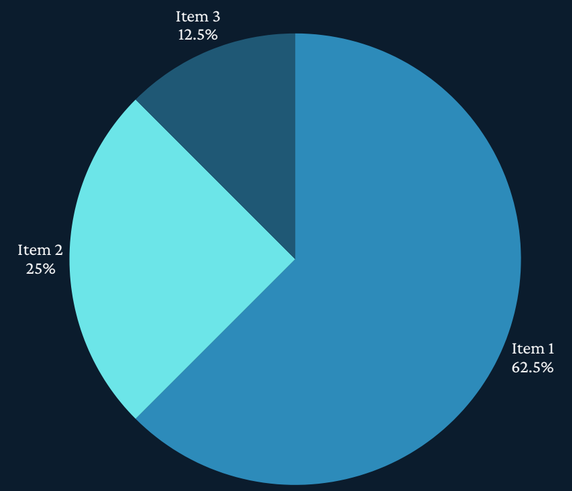
Local files are loaded, normalized, and split into semantic chunks

Chunks are embedded locally using Ollama models (e.g., nomic-embed-text)

No data leaves your environment during ingestion or embedding

STORAGE LAYER: VECTOR + GRAPH

PostgreSQL with pgvector stores semantic embeddings, enabling fast similarity search across chunked documents. GraphDB stores entities, facts, dependencies, ownership, topology, and service relationships extracted during ingestion. Together, these two stores power hybrid retrieval — combining embedding-based similarity with structured graph traversal. This dual-layer approach provides stronger technical context than vector search alone, allowing ProjectRAG to answer questions that require dependency reasoning, not just semantic matching.



GRAPH-AWARE REASONING

ProjectRAG extracts relationships across ingested documents automatically

Graph facts reveal dependencies, ownership, incidents, and service topology

Answers requiring dependency traversal go beyond similarity search alone

Linked entities expose hidden connections across runbooks and architecture notes

ASK A TECHNICAL QUESTION

Query: "Which services are impacted if this network segment changes?"
The query is routed through four stages: vector retrieval surfaces semantically similar chunks, graph context traverses service dependencies and topology, memory injects prior validated facts, and validation checks the answer against retrieved evidence before returning a grounded, traceable response.

LANGGRAPH ORCHESTRATION

Step 1: FastAPI receives and routes the incoming query

Step 2: LangGraph coordinates vector retrieval and graph lookup

Step 3: Memory retrieval and validation are applied in sequence

Step 4: Answer synthesis — explicit, testable, and replaceable

VECTOR RETRIEVAL PATH

Query is embedded locally using Ollama models — no external API calls

pgvector performs semantic similarity search across all stored document chunks

Top-ranked chunks are retrieved based on embedding distance scores

Retrieved chunks serve as grounded document evidence for answer synthesis

GRAPH RETRIEVAL PATH

Entities and relationships are queried directly from GraphDB
The system traverses dependencies and semantic links across services
Connects documents, incidents, ownership, and architecture facts
Enables multi-hop reasoning beyond simple similarity search

MEMORY & HISTORICAL CONTEXT

Memory adds prior project context and previous validated facts
Preserves continuity across repeated technical analysis sessions
Memory must remain scoped, controlled, and auditable
Safe boundaries prevent context bleed across unrelated queries

VALIDATION & EVIDENCE

Answers are checked against retrieved evidence before output

Validation reduces hallucination and improves reviewability

Output includes supporting context and source citations

Confidence signals indicate reliability of each answer

HYBRID ANSWER SYNTHESIS

Stream 1: Vector results — semantically similar document chunks

Stream 2: Graph facts — dependencies, ownership & topology links

Stream 3: Memory — prior context & previously validated facts

Stream 4: Validation → grounded, traceable final answer

COMPLETE ARCHITECTURE

Files → Chunking → Ollama Embeddings → PostgreSQL pgvector → GraphDB
LangGraph orchestrates vector retrieval, graph lookup, memory, and validation
FastAPI serves the query interface and returns validated, evidence-grounded answers
Background jobs handle ingestion retries and idempotent re-processing
Evaluation datasets and broad regression tests form the production quality foundation

CONCLUSION

ProjectRAG delivers controlled, explainable, and testable reasoning over internal technical documents — a strong private engineering knowledge platform. Production upgrades include enforced auth, RBAC, tenant isolation, distributed tracing, metrics dashboards, connector safety, regression testing, and deployment packaging. From solid MVP to production-ready platform.